Offline Shielded Zcash Payments: Technical Spec

Oct 1, 2026 · @binaryy

Summary

We are building an Android wallet that sends shielded Zcash payments with no internet: the phone signs the payment offline and nearby phones relay it over Bluetooth until one reaches the network. Target: ZECATHON, submissions due Oct 28, 2026. Primary track: Shielded Payments; Wildcard (which names hardware) is the fallback fit.

The pitch in one line: private money that keeps moving when the internet is shut off, where the strangers relaying it learn nothing.

Problem

Every crypto wallet stops working the moment the internet goes down, which is exactly when private money matters most. Governments cut connectivity during protests and elections, and the same moments often bring frozen bank accounts. People already reach for offline tools in these moments: BitChat, a Bluetooth mesh messenger, has been widely used to get around local shutdowns. But it moves messages, not money.

Three things make this painful today:

- **No offline path for Zcash.** The idea was floated on the Zcash forum in late 2024; we found no implementation.
- **Existing offline payment experiments leak data.** An offline UPI mesh prototype had to bolt on its own encryption so relaying phones could not read payments.
- **Unreliable connectivity is normal, not rare.** Markets, transit, rural areas and crowded events all have dead zones where a payment simply cannot be sent.

Solution

The wallet builds and signs a complete shielded Ironwood transaction fully offline, hands it to the receiver over Bluetooth, and lets any nearby phone carry it to the network. The transaction is set to never expire, so it stays valid however long the relay takes.

Why Zcash specifically. A shielded transaction is already an encrypted blob: sender, receiver and amount are hidden. Relaying phones carry it without learning anything, so privacy is not an add-on here; it is what makes stranger-to-stranger relaying safe.

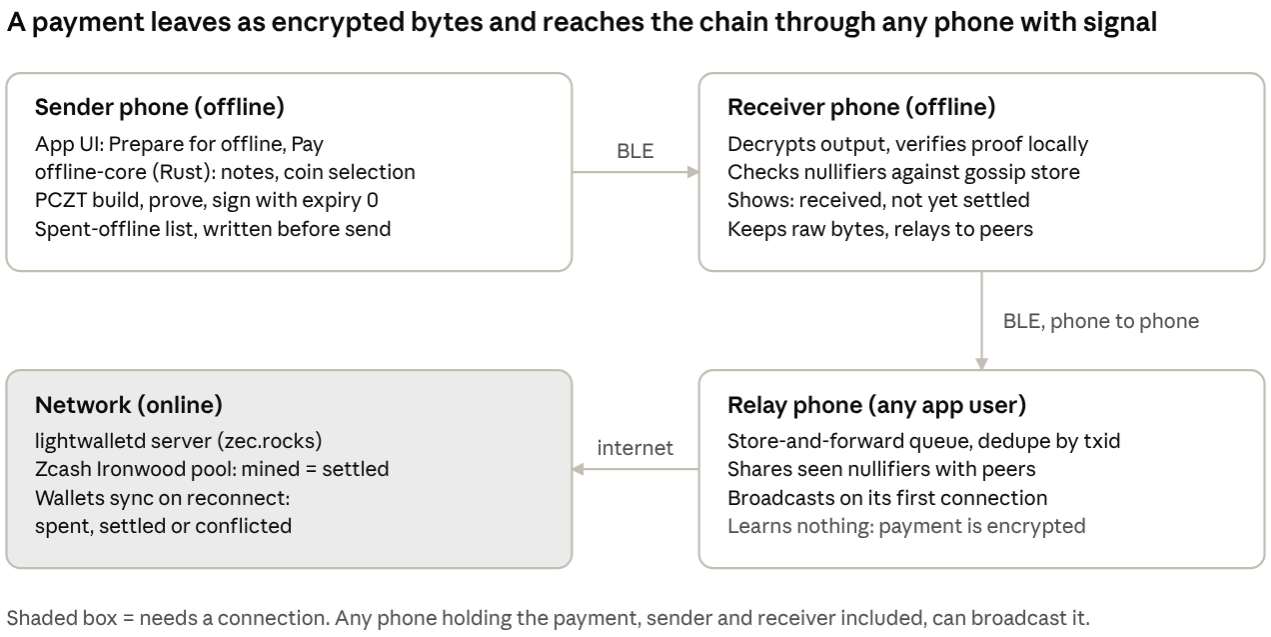
What the user experiences:

1. **Prepare for offline** (while connected): choose how much to carry as offline cash; one tap splits it into cash-like denominations.
2. **Pay offline**: pick an amount, the phone signs, the receiver's phone gets it over Bluetooth and shows "received, not yet settled."
3. **Relay**: both phones, plus any phone running the app nearby, carry the payment until one gets a connection and broadcasts it.
4. **Settle**: once mined, both wallets flip the payment to "settled."

The demo moment: on stage, put a phone in airplane mode, pay someone, and watch the payment hop through a second phone and land on mainnet.

Architecture

Three phone roles share one app and one Rust core; only the last hop needs the internet.



system architecture · 3 phones, 1 network

The sender signs offline and hands raw transaction bytes to the receiver over Bluetooth. Every phone that holds the payment queues it, gossips its nullifiers, and broadcasts it the moment it gets a connection.

Core flows

Five flows cover the whole product; each step below is what the code must do, in order.

1. Prepare for offline (online)

1. Sync the wallet to the chain tip so every note has a fresh witness (its position in the note tree).
2. The user chooses how much to prepare as offline cash on a keypad. It starts at everything not yet offline, so a plain tap splits it all; Max resets it.
3. Split that amount into denominations in one self-send, e.g. two each of 2, 1, 0.5, 0.2, 0.1, 0.05, 0.02, 0.01 ZEC (about 16 to 20 notes). Any rest below 0.01 ZEC becomes one extra note, so the pocket gets the exact amount.
4. Wait for that transaction to be mined, then sync again so the new notes have witnesses.
5. Store, per note: value, nullifier, witness, anchor height, status `available`.

Offline cash is a pocket: only pocket notes are spent offline. A note is in the pocket when one of our own transactions made it (a split, or an offline payment's change) and it is not a partial split's leftover change. The SDK marks every self-payment output as change, so a partial split tags each pocket output with the memo `offline-cash`. While the SDK builds a split, pocket notes are locked so it cannot spend them. Funds from anyone else (a faucet, other phones) stay "in wallet" until prepared; Home shows both.

Fee check (ZIP 317: 5,000 zatoshis per action, minimum 2): a 20-output split costs about 0.001 ZEC.

2. Pay offline

1. Receiver's phone shows its shielded address as a QR code or Bluetooth advert, plus the amount.
2. Sender's app picks notes summing exactly to the amount where possible (coin-change selection over `available` notes).
3. Build the transaction as a PCZT: Ironwood spends, receiver output, change output only if exact change is impossible, expiry height 0, ZIP 317 fee.
4. Prove and sign on the phone, then extract the final transaction bytes.
5. **Before sending:** write each spent note's nullifier and the raw transaction bytes to local storage, status `spent-offline`.
6. Send the bytes to the receiver over Bluetooth.

3. Receive offline

1. Trial-decrypt the transaction's outputs with the receiver's viewing key; confirm the amount and address.
2. Verify the proof and signatures locally (no network needed).
3. Check the nullifiers against the gossip store (flow 5); a match means "conflict seen."

4. Store the raw bytes and show "received, not yet settled."

4. Relay and settle

1. Every phone running the app forwards pending transactions to peers it meets (store-and-forward, de-duplicated by txid).
2. Any phone that gets a connection broadcasts every pending transaction it holds to a lightwalletd server.
3. On reconnect, each wallet syncs: a nullifier seen on chain marks its note `spent`; a received output seen on chain marks the payment `settled`.
4. The sender's own pending transactions are rebroadcast by the sender too. A signed note is never re-spent in a new transaction, even if the old one seems lost.

5. Change handling

- **Pending change note:** if exact change is impossible, the change output belongs to the sender but stays unspendable until the transaction is mined.
- **Receiver gives change:** for the demo, the receiver can pay the difference back offline from its own notes, like cash.

Double-spend model

Offline double-spending cannot be fully prevented without trusted hardware, so we prevent honest mistakes, detect cheaters fast, and cap the damage. Every offline payment is a race to the chain: the honest payment crawling through the mesh versus any conflicting spend the sender makes, for example by walking to a signal and spending the same note from another wallet restored from the same seed.

Layer	What it does	Stops	Does not stop
Refuse to sign twice	Local `spent-offline` list, written before the Bluetooth send; note picker skips listed notes	Accidental reuse, crashes mid-send	A modified app, or the same seed in another wallet

Layer	What it does	Stops	Does not stop
Nullifier gossip	Phones share the nullifiers of every transaction they relay; two transactions with one nullifier trigger a mesh-wide alert. Nullifiers reveal nothing about the owner	Cheating going unnoticed within mesh range	The cheat itself, or a conflicting spend broadcast far away
Limits and labels	Per-payment and per-session offline caps; "received, not yet settled" until mined	Large losses	Small losses
Fast settlement	Receiver relays to every peer and broadcasts on the first connection	Slow honest payments losing the race	A cheater who reaches the network first

What we claim: honest payments settle reliably; cheating is detected locally and bounded in size. **What we do not claim:** that double-spending is solved. The analogy for judges is offline card terminals on aircraft: accepted offline, small amounts, risk carried by the merchant.

User-facing rule: never use one wallet seed on two devices while offline.

Tech stack and protocol facts

A native Android app in Kotlin, with all Zcash logic in Rust via librustzcash, called through a thin JNI bridge.

Layer	Choice	Why
Platform	Android, Kotlin	Background Bluetooth needs a native app; iOS restricts it further; cheap demo devices
Wallet sync and storage	zcash-android-sdk 3.0.1-SNAPSHOT (Ironwood)	Used in production by Unstoppable and Zapp
Offline transaction building	`pczt` + `zcash_client_backend` (Rust)	The SDK builds and broadcasts at once; PCZTs separate build, prove, sign and send
Reference implementation	zcash-devtool `pczt create/prove/sign/combine/send`	Already implements the offline-signing flow we need
Mesh transport	BLE GATT + Wi-Fi Direct fallback; study BitChat's open-source Android code	Proven phone-to-phone mesh
Broadcast	lightwalletd (zec.rocks testnet and mainnet)	Standard light-client endpoint
Local store	SQLite: notes, nullifiers, raw tx bytes, gossip set	Survives crashes; raw bytes avoid a known reconstruction bug

Protocol facts we rely on

Fact	Source	Status
Expiry height 0 means the transaction never expires	ZIP 203	Confirmed
Anchor may be any earlier block's note-tree root	Protocol spec	Spec says so; confirm in spike 1
Fee = 5,000 zatoshis × max(2, actions); Ironwood actions count	ZIP 317	Confirmed
One transaction can spend many notes	Protocol design	Confirmed
v6 allows signing now, setting anchor and proofs later	librustzcash #2529	Confirmed; stretch goal only

Fact	Source	Status
Unmined v6 reconstruction bug only hits non-zero expiry	librustzcash #2987	Does not affect expiry-0 transactions

Design limit: a transaction with hundreds of outputs exceeds practical size, so denominations stay around 20 notes.

Verification spikes

Four unknowns could kill the project, so they are tested before any app or UI work; spikes 1 and 2 decide go or no-go by Oct 4.

#	Unknown	Test	Pass	If it fails
1	A transaction built offline against an older anchor, with expiry 0, is accepted hours later	zcash-devtool on Ironwood testnet: sync, disconnect, `pczt create` → `prove` → `sign` → `combine`, save; wait 4+ hours of blocks; `pczt send`	Transaction confirms	Find the anchor age window in Zebra's code; cap offline time to fit it
2	zcash-devtool can set expiry 0 and handles Ironwood	`pczt create --help`; inspect the created PCZT	Option exists, or a small patch adds it	Patch the builder call in our own fork
3	The PCZT flow runs on Android	Cross-compile the Rust crates for `aarch64-linux-android`; run create/prove/sign on a phone in airplane mode	Signed bytes produced on device; proving time under 30 s	Fall back to SDK internals; reduce actions per transaction
4	Bluetooth relays a 10 to 20 KB blob (a few-action transaction) reliably	3 Android phones, different brands, screens off, phones in pockets	95%+ delivery within 60 s at 1 hop; 2-hop relay works	Foreground service + Wi-Fi Direct; demo with screens on

Record results in this doc as each spike finishes.

Settlement news over the mesh

When any phone sees an offline payment mined, phones that are still offline learn it over Bluetooth, so a receiver can see "settled" without internet. Today the mesh carries payments, inventories and nullifier claims, but a phone marks a payment `settled` only after seeing it mined itself.

An offline phone cannot check the chain, so a bare "it's settled" from a nearby phone is only a claim. The work comes in two stages: first pass the claim on and label it as one, then carry a proof the offline phone can check itself.

Stage	What travels	What an offline phone may do	Trust
1. Settlement claims	`Settled{txid, height, block hash}`, gossiped with the inventory, bounded and newest first, like nullifier claims	Show "a nearby phone reports this settled at block N". Never move money into the balance or clear a pending payment	A hint. A dishonest phone can lie; nothing is counted on it
2. Proof of inclusion	The claim plus the block header and the transaction's Merkle path to the header's transaction root	Check the path, the header's proof of work and that it chains from a header seen at its last sync; then treat the payment as settled at that height	Proof, as in SPV light wallets, subject to a confirmation-depth rule

Rules

- **Claims never move money.** A receiver's "received, not yet settled" line moves into the balance only on its own sync (stage 1) or a checked proof (stage 2).
- **Contradictions raise the conflict alert.** A claim that another transaction spending the same nullifier was mined turns the existing red alert on and marks the payment conflicted.
- **Forged claims are dropped.** An unknown txid or a height below what the phone already knows is ignored and never relayed.
- **Nothing new is revealed.** A claim names only a txid and a block, which the relays already hold.

Tests

- [ ] A pays B offline; C, online, settles it. A and B stay offline and show the "reported settled" hint within 60 s over Bluetooth. Their balances do not change until their own sync.

- [ ] A forged claim (unknown txid, wrong height) is ignored and never relayed; balances do not change.
- [ ] Stage 2: a bad Merkle path, a header without enough work, or one that does not chain is rejected; a good proof marks the payment settled with no sync.

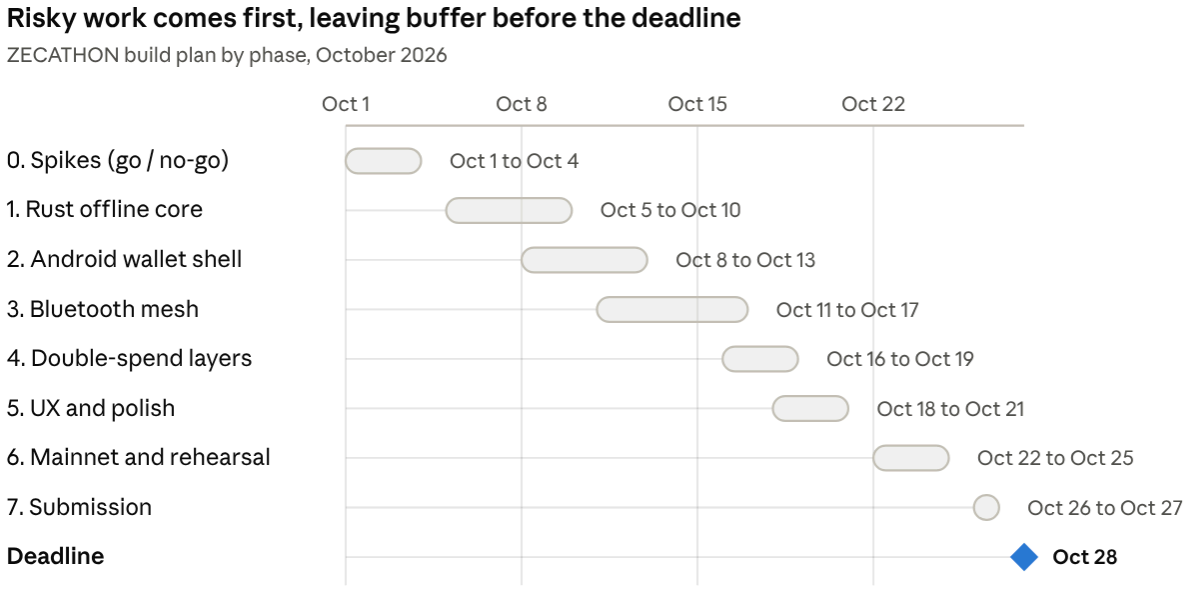
Open questions

- [ ] Cost of checking Zcash headers on a phone (Equihash solution check) and how many headers make the work meaningful.
- [ ] Confirmation depth before a proven payment counts in the balance.
- [ ] How the UI tells "settled, proved by a nearby phone" apart from "settled, seen by this phone".

Build order: stage 1 is a Phase 4 add-on (should ship); stage 2 is a stretch goal before Phase 6. Both need working Bluetooth between test devices: two real phones, or emulators once netsim is fixed (broken as of Oct 23). Added Oct 24, 2026.

Build order

Seven phases over 28 days, risk first: spikes by Oct 4, a working offline payment on testnet by Oct 13, the full mesh by Oct 19, mainnet demo by Oct 25, leaving three days of buffer before the Oct 28 deadline. Each phase ends in something runnable.



build timeline · 8 phases to Oct 28

Phase 0: Spikes (Oct 1 to 4)

- [ ] Install Rust, clone zcash-devtool, create an Ironwood testnet wallet, fund it from a faucet
- [ ] Spike 2: confirm or patch expiry-0 support in `pczt create`
- [ ] Spike 1: build and sign offline, wait 4+ hours, broadcast, confirm
- [ ] Spike 4: BLE blob test on 3 phones (runs in parallel; no Zcash needed)
- [ ] **Gate:** spikes 1 and 2 pass, or the anchor window is known and acceptable

Phase 1: Rust offline core (Oct 5 to 10)

One Rust crate, `offline-core`, tested from a desktop CLI before it touches Android.

- [ ] Note store: notes, values, nullifiers, witnesses, status (`available`, `spent-offline`, `spent`)
- [ ] Denomination split: build the self-send that breaks a balance into the 2/1/0.5/0.2/0.1/0.05/0.02/0.01 set
- [ ] Coin selection: exact-sum first, smallest overpay second
- [ ] Offline payment builder: PCZT create, prove, sign, combine, extract bytes; expiry 0; ZIP 317 fee

- [ ] Receive verifier: decrypt outputs with the viewing key, verify proof and signatures, extract nullifiers
- [ ] Spike 3: cross-compile for Android and run the builder on a phone
- [ ] **Gate:** CLI pays testnet offline and the payment confirms after a delayed broadcast

Phase 2: Android wallet shell (Oct 8 to 13)

Starts while Phase 1 finishes; uses the SDK for sync and our crate for offline work.

- [ ] Project setup: Kotlin app, SDK 3.0.1-SNAPSHOT, JNI bridge to `offline-core`
- [ ] Wallet create and restore, sync, Ironwood balance
- [ ] "Prepare for offline" screen: split, wait for mining, show the denomination set
- [ ] Pay and receive by QR code first (no Bluetooth yet): transaction bytes as an animated QR
- [ ] **Gate:** two phones in airplane mode pay each other by QR; either phone broadcasts later and it settles on testnet

Phase 3: Bluetooth mesh (Oct 11 to 17)

- [ ] BLE GATT service and client; chunk and reassemble transaction bytes with a checksum
- [ ] Peer discovery and handshake (app ID, protocol version)
- [ ] Store-and-forward queue, de-duplicated by txid, with a hop limit
- [ ] Connectivity watcher: broadcast every queued transaction on any connection, with retry
- [ ] Android foreground service so relaying survives screen-off
- [ ] **Gate:** phone A pays phone B offline; phone C carries it out of range and broadcasts; it settles

Phase 4: Double-spend layers (Oct 16 to 19)

- [ ] Write-before-send `spent-offline` records; startup recovery after a crash
- [ ] Nullifier gossip: exchange seen nullifiers on every peer contact; conflict alert UI
- [ ] Offline limits per payment and per session; "not yet settled" labels
- [ ] Reconciliation on reconnect: mark spent, settled, or conflicted
- [ ] **Gate:** a deliberate double spend from a second device is flagged on nearby phones within 60 s

Phase 5: UX and polish (Oct 18 to 21)

- [ ] Payment states end to end: pending, relayed, settled, conflicted

- [ ] Receiver-gives-change flow (cut first if behind)
- [ ] Offline balance view: notes available vs locked
- [ ] Seed-on-two-devices warning

Phase 6: Mainnet and rehearsal (Oct 22 to 25)

- [ ] Switch to mainnet with small amounts; run the full flow 10 times, log failures
- [ ] Record the demo video, with block explorer confirmation on screen
- [ ] Field test: market or campus with weak signal

Phase 7: Submission (Oct 26 to 28)

- [ ] README: build steps, architecture, threat model, limits
- [ ] Final video and write-up; submit by Oct 27, keeping Oct 28 as buffer

Cut lines

Priority	Scope
Must ship	Prepare for offline, offline pay, 1-hop Bluetooth, delayed broadcast, settle on mainnet, refuse-to-sign-twice
Should ship	Multi-hop relay, nullifier gossip, offline limits
Could ship	Receiver gives change, field test, v6 pre-signed change chains

Demo and submission plan

The demo is a three-phone story on mainnet in under three minutes, ending on a block explorer showing the confirmed transaction.

1. **Setup (20 s):** "The internet is down. Ada sells bread; Tunde wants to buy." Phones A and B go into airplane mode on camera.
2. **Pay (30 s):** Tunde pays 0.05 ZEC; Ada's phone shows "received, not yet settled."
3. **Relay (40 s):** phone C walks over, picks up the payment over Bluetooth, walks out of range, reconnects and broadcasts.
4. **Settle (30 s):** Ada's payment flips to "settled"; cut to the block explorer showing an Ironwood transaction with nothing readable.

5. **The cheat (40 s):** Tunde tries to spend the same note again from a second device; nearby phones flag the conflict, and we explain the limits.

Pitch framing for judges: "Offline double-spending can't be fully prevented without trusted hardware. We prevent honest mistakes, detect cheaters within minutes, and cap the damage, while the strangers carrying your money learn nothing."

Submission package: public repo, README with threat model, demo video, and this spec.

Risks and open questions

Risk	Likelihood	Fallback
Anchor age window too short for long offline periods	Low	Cap offline time; re-prepare before going offline
Ironwood support in the SDK snapshot is rough	Medium	Use `offline-core` for more of the wallet; pin a known-good commit
Background BLE throttled on some phones	High	Foreground service; demo with screens on; Wi-Fi Direct
Proving too slow on low-end phones	Medium	Fewer actions per payment; test on the demo phones early
Judges see double-spending as a dealbreaker	Medium	Lead with the threat model; show the conflict alert live

Open questions:

- [ ] Exact submission time and format on Oct 28 (check the ZECATHON dashboard rules)
- [ ] Whether ZECATHON rules require mainnet, open-source licensing, or a video length
- [ ] Project name

Sources

- ZECATHON
- ZIP 203: Transaction Expiry
- ZIP 317: Proportional Transfer Fee Mechanism
- Zcash Protocol Specification
- zcash-devtool walkthrough
- librustzcash #2529: pre-signed v6 transaction chains
- librustzcash #2987: unmined v6 reconstruction bug

- Unstoppable Wallet grant (Ironwood on Android)
- Zapp for Android
- Zcash forum: paying offline with Zcash
- Offline UPI mesh prototype
- BitChat